

Plataforma Distribuída

para Acompanhamento de Manutenção Veicular

Arquitetura orientada a eventos com RabbitMQ • Edge + Cloud

XRSC09 — Sistemas Distribuídos • Projeto Final

Grupo XX — Junho de 2026

Membro 1 • Membro 2 • Membro 3 • Membro 4 • Membro 5

O problema

Transparência na manutenção veicular

Cenário sorteado:

Empresa de manutenção de veículos quer permitir que o cliente acompanhe o serviço em tempo real, com etapas como relato, diagnóstico, identificação da causa, execução do reparo e substituição rastreável de peças.

Dor central

- Cliente não vê o que foi feito → desconfiança e reclamações
- Sem evidência verificável de cada etapa do serviço
- Peças substituídas sem rastreio claro
- Comunicação informal e fragmentada entre cliente e oficina

Por que uma solução distribuída?

Limites do monolito

Escala desigual

Processar vídeo escala diferente de servir REST. Tudo no mesmo processo amarra as cargas.

Vários destinatários

Mudar etapa gera notificação, auditoria, dashboard e e-mail — fan-out natural.

Resiliência geográfica

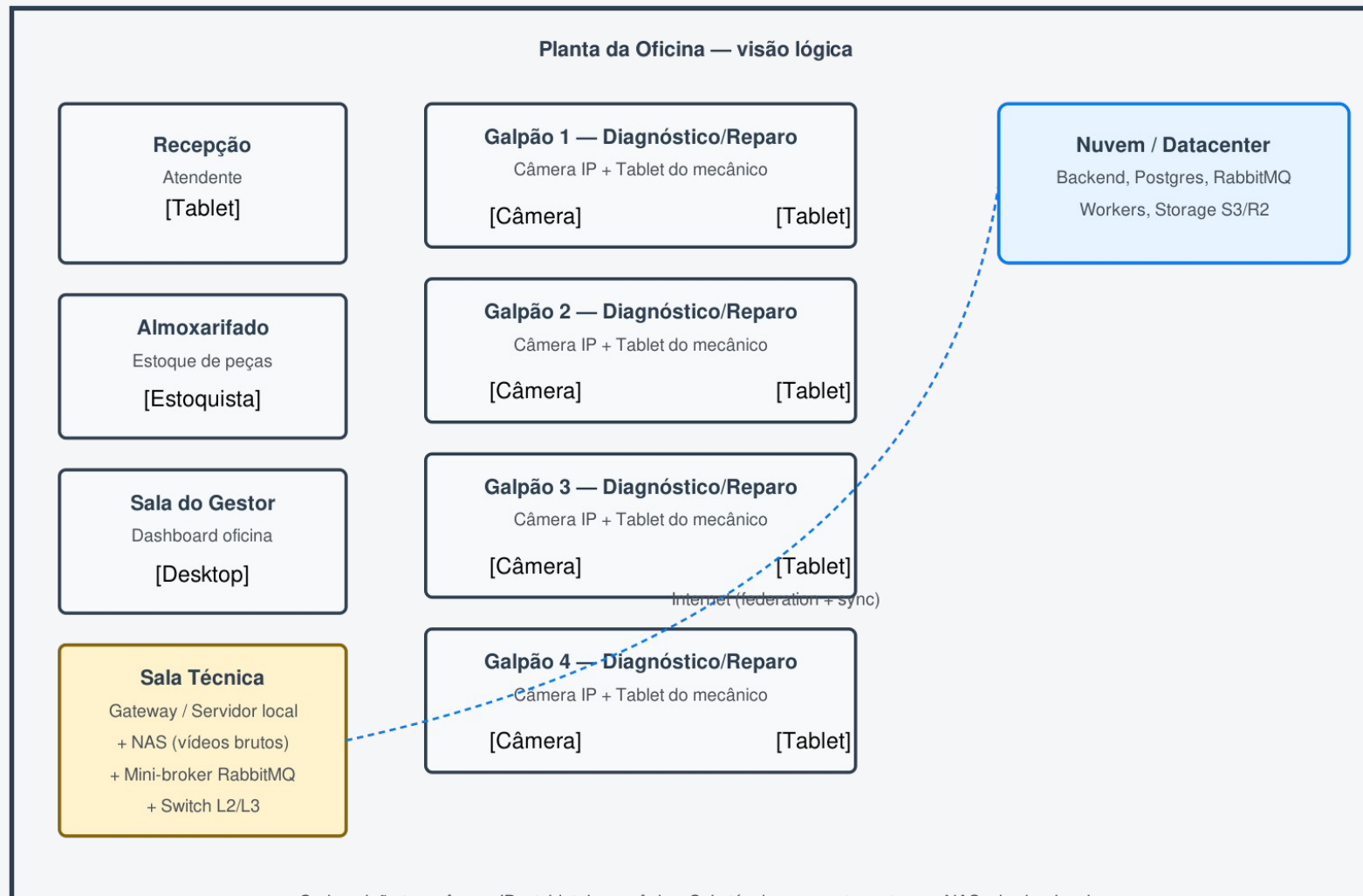
A oficina precisa registrar evidência mesmo offline e sincronizar depois.

Extensibilidade

Novos consumers (billing, relatório, seguradora) devem entrar sem mexer no núcleo.

Planta da oficina

Onde ficam câmeras, terminais e o gateway local



Cada galpão tem câmera IP e tablet do mecânico. Sala técnica concentra gateway, NAS e broker local.

4 galpões com câmera IP + tablet • almoxarifado • recepção • sala técnica com gateway, NAS e mini-broker

Modelo C4 — Nível 1: Contexto

Atores e sistemas externos

C4 Nível 1 — Diagrama de Contexto



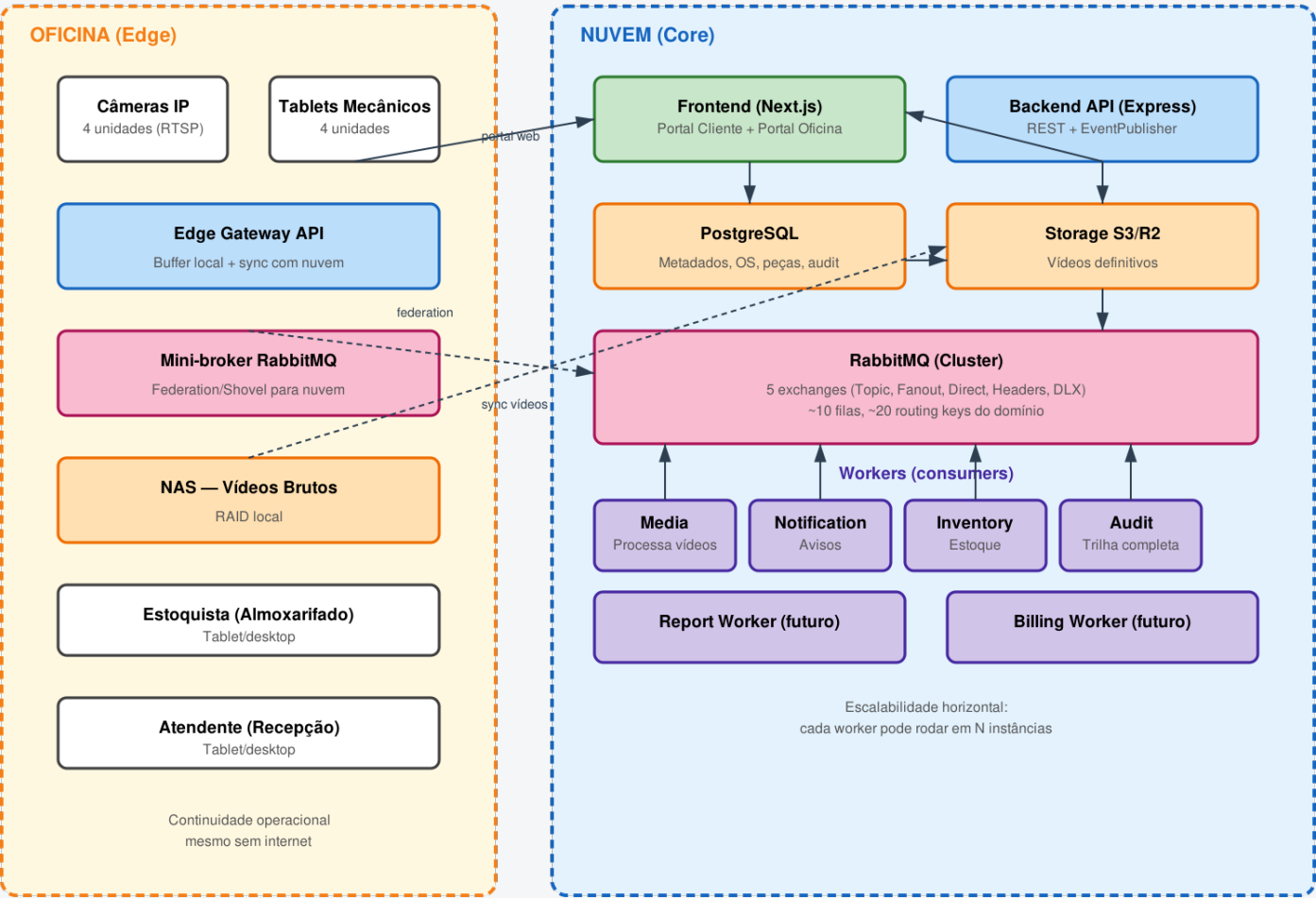
Atores interagem via portal web. Sistema integra-se a câmeras, storage, gateways de notificação e sistemas externos.

5 atores • integração com câmeras, storage, gateways, ERP, pagamento, NF-e

Modelo C4 — Nível 2: Containers

Zona de borda (oficina) + zona de nuvem

C4 Nível 2 — Diagrama de Containers

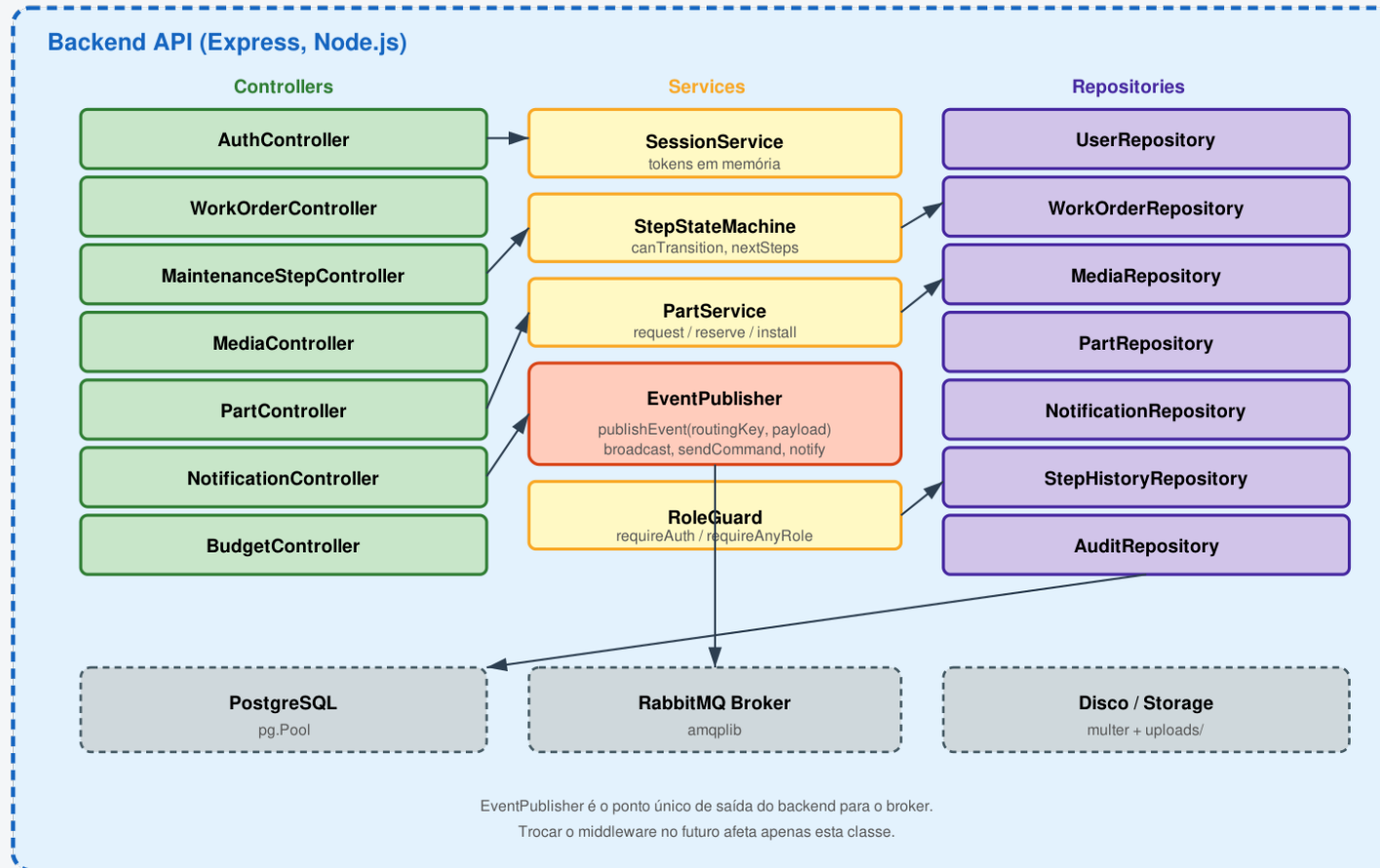


Edge: gateway, NAS, mini-broker federado • Cloud: API, Postgres, RabbitMQ cluster, workers, storage

C4 Nível 3 — Componentes do Backend

Controllers • Services • Repositories • EventPublisher

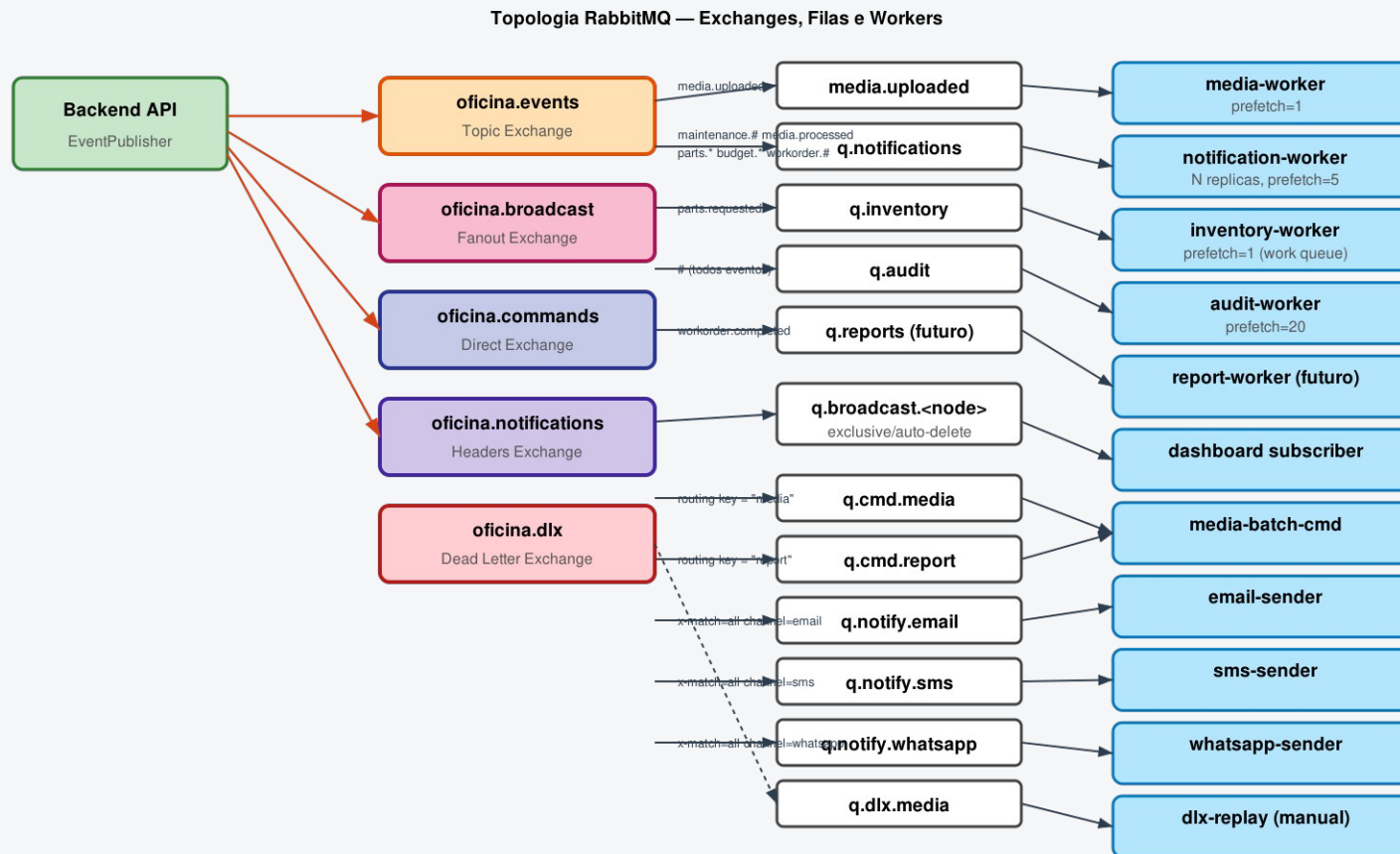
C4 Nível 3 — Componentes do Backend API



EventPublisher é o ponto único de saída para o broker — substituir middleware afeta só essa classe

Topologia RabbitMQ

5 exchanges • ~12 filas • ~25 routing keys planejadas



5 exchanges (Topic, Fanout, Direct, Headers, DLX) + ~12 filas no protótipo + ~25 routing keys planejadas.

No protótipo demonstramos os 4 padrões principais. Os demais ficam descritos como evolução.

Padrões em uso: Pub/Sub, Topic Routing, Work Queue, Headers, DLX, Federation (planejada)

Topic + Fanout + Direct + Headers + DLX • padrões Pub/Sub, Work Queue, Topic Routing, Headers, DLX

Cinco exchanges, cinco padrões

Cada tipo de AMQP cobre um caso de uso

oficina.events

Topic

Hub principal — routing key dominio.entidade.acao

oficina.broadcast

Fanout

Avisos globais (manutenção da plataforma, alarmes)

oficina.commands

Direct

Comandos diretos para um worker específico

oficina.notifications

Headers

Roteia notificação por canal (email/sms/whatsapp)

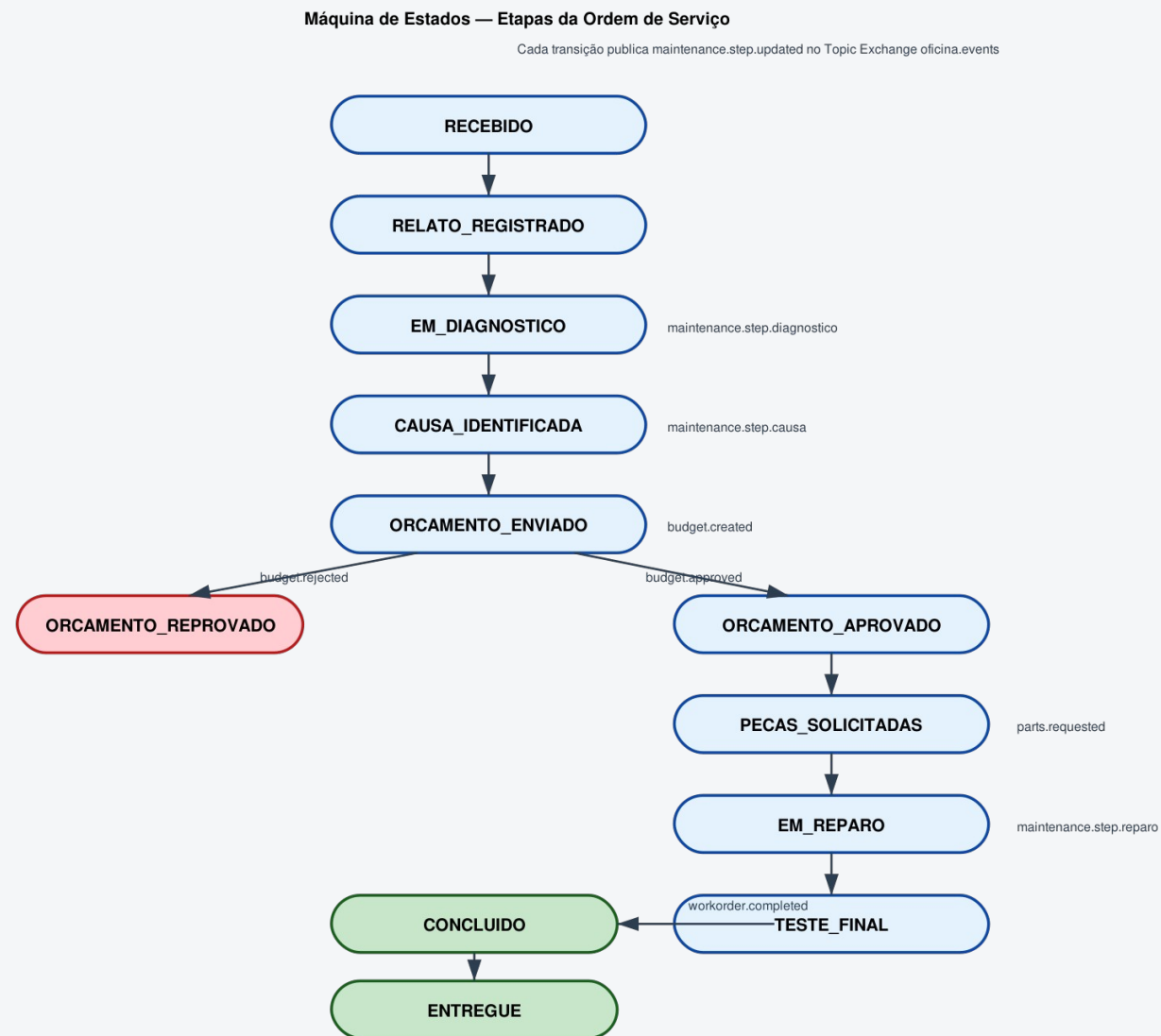
oficina.dlx

DLX

Dead Letter — mensagens com falha para inspeção/retry

Máquina de estados das etapas

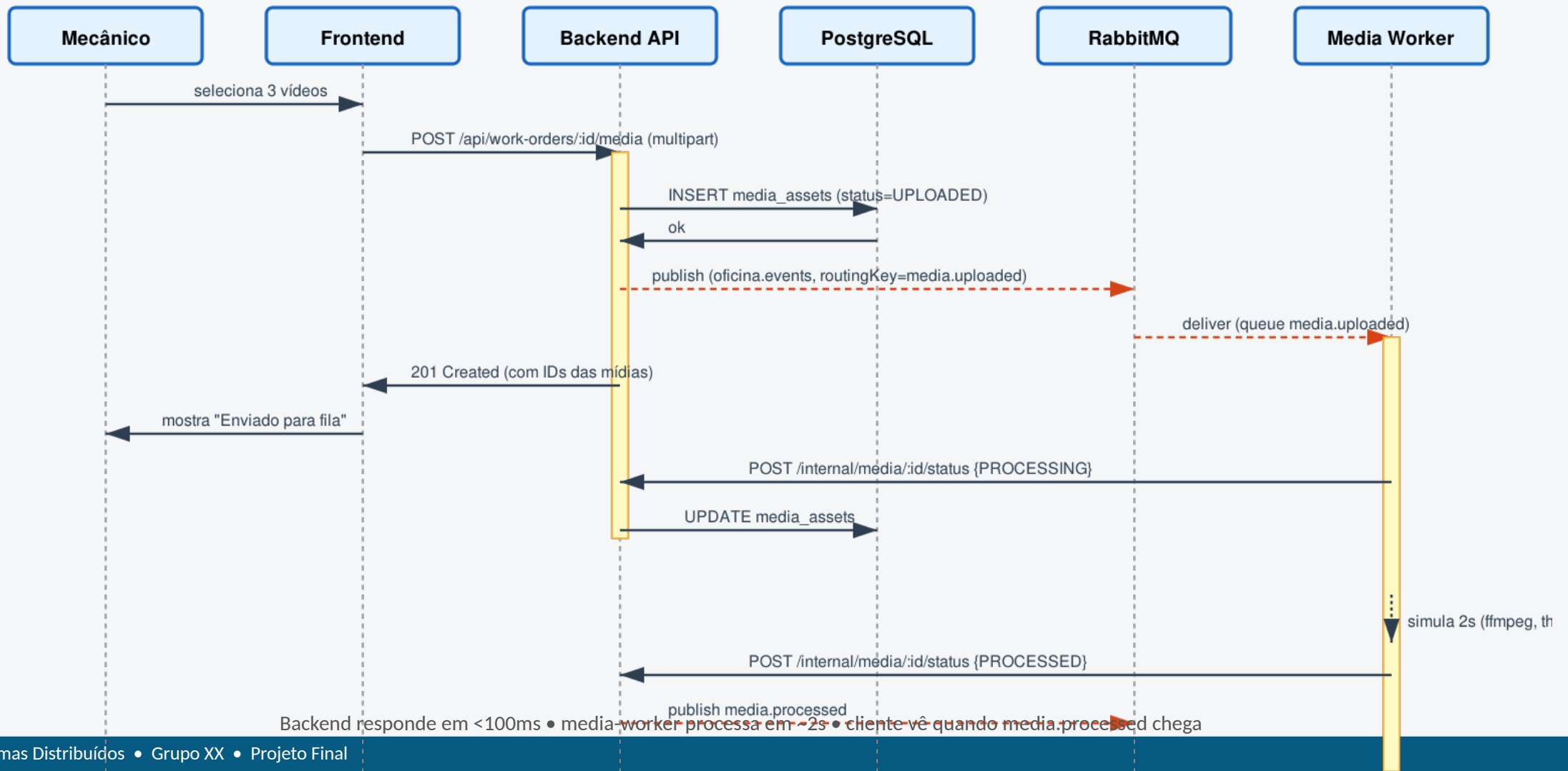
12 estados, transições validadas, evento por transição



Fluxo 1 — Upload de mídia

Resposta síncrona rápida • processamento assíncrono

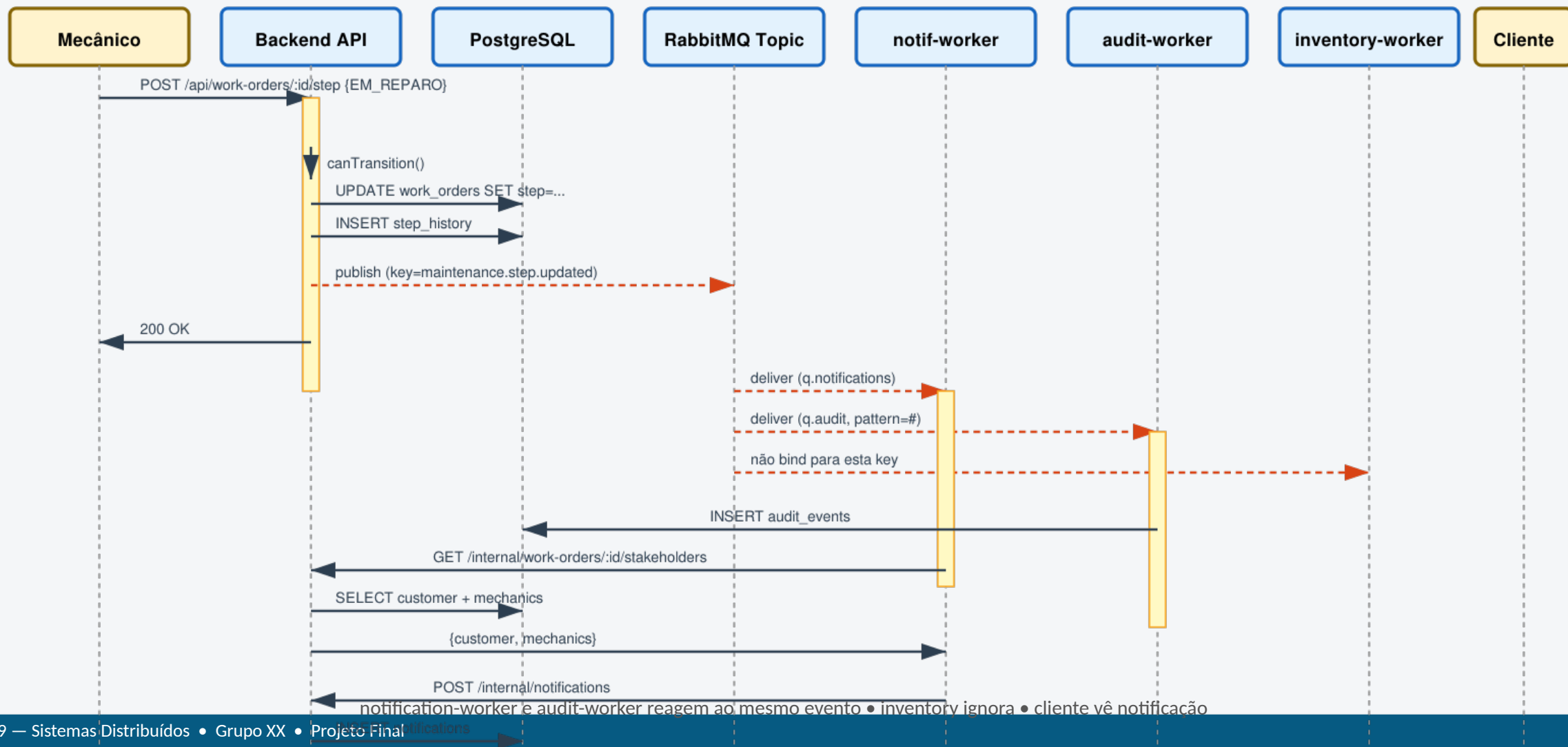
Sequence Diagram — Upload de mídia com processamento assíncrono



Fluxo 2 — Mudança de etapa

Uma chamada → cascata de workers em paralelo

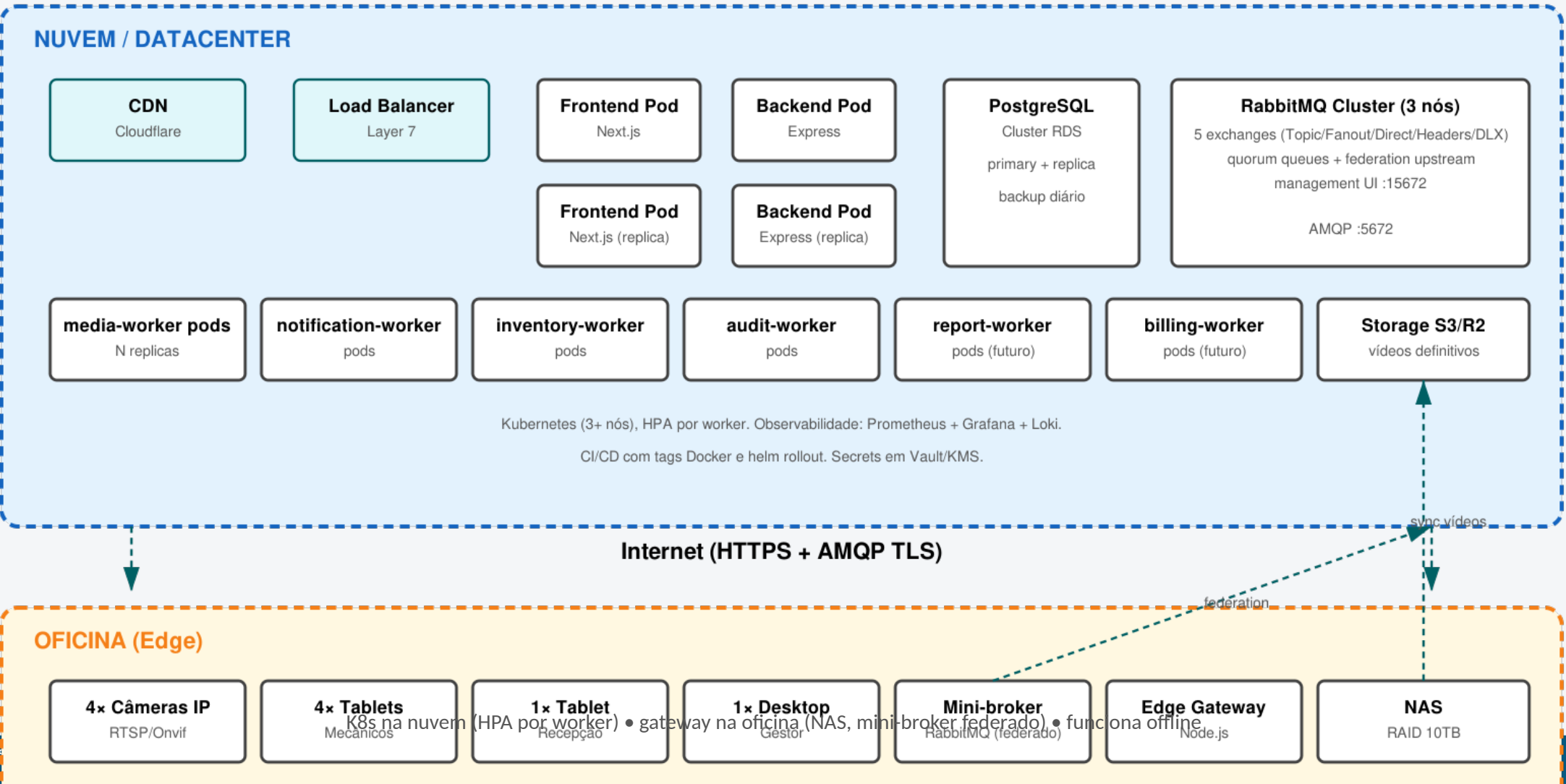
Sequence Diagram — Mudança de etapa e cascata de notificações



Implantação híbrida edge + cloud

Resiliência operacional como decisão arquitetural

Diagrama de Implantação Híbrido (Edge + Cloud)



Stack do protótipo

Tudo containerizado, sobe com um único comando

Frontend

Next.js 14 + React 18

Backend

Node.js 20 + Express 4

Broker

RabbitMQ 3 (management UI)

Banco

PostgreSQL 16

Workers

4× Node + amqplib

Orquestração

Docker Compose

\$ docker compose up --build

O que o protótipo demonstra

Fatia mínima viável da arquitetura completa

Implementado no código:

- 5 exchanges (Topic, Fanout, Direct, Headers, DLX)
- Filas com bindings por routing key, # e *
- Máquina de estados de 12 etapas, transições validadas
- Upload de mídia + processamento assíncrono
- Rastreamento de peças (request → reserve → install)
- Notificações personalizadas para stakeholders
- Trilha de auditoria com binding universal #
- Frontend autenticado por papel (4 perfis)

Defendido na apresentação:

- Arquitetura edge-cloud com mini-broker federado
- Gateway local com NAS para vídeos brutos
- Cluster RabbitMQ com quorum queues em K8s
- HPA por worker, observabilidade Prometheus/Grafana
- Integração real com email/SMS/WhatsApp (Headers)
- Storage S3/R2 para vídeos definitivos
- Report worker, billing worker, NF-e
- Push em tempo real via WebSocket

Cenários executados

Quatro fluxos end-to-end com a stack levantada

C1 — Mudança de etapa

Backend → maintenance.step.updated → notification + audit + portal

<500 ms

C2 — Upload de mídia

3 arquivos → media.uploaded → worker → media.processed → notif

~7 s p/ 3 arquivos

C3 — Solicitação de peça

parts.requested → inventory-worker → reserve → parts.reserved

~1 s

C4 — Auditoria universal

audit-worker assina '#' → 11 eventos capturados nos cenários 1+2+3

100% cobertura

Limitações e trabalhos futuros

O que o protótipo não cobre — e como evoluir

Limitações conhecidas:

- Sessão em memória (impede escala do backend)
- Processamento de mídia é simulado (sem FFmpeg)
- Edge gateway só na apresentação
- Sem WebSocket — cliente faz polling manual
- Sem integração real com email/SMS/WhatsApp
- Sem upload para S3/R2

Trabalhos futuros:

- Implementar edge gateway com federation
- Quorum queues no broker
- Push em tempo real via WebSocket / SSE
- Integração real com gateways via Headers Exchange
- FFmpeg para thumbnail + transcodificação
- Report worker (PDF da OS concluída)
- Prometheus + Grafana para observabilidade
- Integração com NF-e e billing real

Demo ao vivo

`docker compose up`

- frontend <http://localhost:3000>
- backend <http://localhost:4000/health>
- RabbitMQ Management <http://localhost:15672> (guest/guest)

O que entregamos:

- Projeto completo de sistema distribuído orientado a eventos para um problema real e atual
- Modelo C4 nos 4 níveis + planta + máquina de estados + 2 sequence diagrams + deployment
- Protótipo funcional com 5 exchanges, 4 workers, 9 tabelas, frontend autenticado, Docker Compose

Lições aprendidas:

- Desacoplar via broker é o que viabiliza escalar workers de forma independente
- A trilha de auditoria 'de graça' pelo padrão # é uma vantagem técnica concreta do Topic Exchange
- A arquitetura edge-cloud não é luxo: é o que mantém a oficina operacional durante quedas
- Centralizar a publicação em EventPublisher isola a decisão de middleware

Obrigado.

Perguntas?

`github.com/<usuario>/projeto-final-sd`